CIFAR-10 ML System Optimization

Performance Evaluation of DataLoader Workers and Batch Size

Project Report

Student / Group: _____

Course: ML System Optimization

Date: 28 October 2026

Abstract

This project measures the effect of DataLoader worker count and batch size on a compact CNN trained on CIFAR-10. Experiment A compares 0, 2, and 4 workers; Experiment B compares batch sizes 64, 128, and 256 with four workers. Each setting used three trial seeds (42, 43, 44) and 15 epochs. Four workers reduced mean steady-state epoch time by 38.44% versus two workers and increased throughput by 62.89%. Batch size 256 reduced mean epoch time by 10.77% versus batch size 64 and increased throughput by 11.23%, while batch size 64 achieved the highest mean best-validation accuracy. This is single-GPU tuning, not multi-GPU distributed scaling.

1. Problem Statement and Objectives

The objective is to measure the effect of selected system settings on training time, throughput, and validation performance.

- Establish a baseline training run.
- Compare worker counts with other major settings held constant.
- Compare batch sizes with four workers.
- Report mean and sample standard deviation over three seeds.
- Discuss performance–accuracy trade-offs and limitations.

2. Hardware and Software Environment

Component	Recorded environment
OS	Windows 10
Python	3.11.9
PyTorch / Torchvision	2.5.1+cu121 / 0.20.1+cu121
GPU	NVIDIA RTX 4050 Laptop GPU, 6 GB
CUDA	Available; one physical GPU

Rerun src/check_environment.py before submission to confirm the environment.

3. Dataset and Data Pipeline

CIFAR-10 contains 60,000 colour images of size 32×32 across 10 classes. The recorded split uses 45,000 training images, 5,000 validation images, and the official 10,000-image test set. Split seed is 42; trial loader seeds are 42, 43, and 44. Confirm the transforms and split in src/dataset.py.

4. Model and Training Configuration

SmallCNN uses convolutional blocks 3→32, 32→64, and 64→128 with batch normalization, ReLU, and max pooling; then flattening, a 256-unit fully connected layer, ReLU, dropout 0.3, and a 10-class output.

Setting	Value
Loss	CrossEntropyLoss
Optimizer	Adam
Learning rate / weight decay	0.001 / 0.0001
Epochs / trial seeds	15 / 42, 43, 44
Experiment A batch size	128
Experiment B workers	4

5. Experimental Methodology

Baseline. Batch size 128, two workers, seed 42, 15 epochs. Best validation accuracy 80.46%, test accuracy 78.99%, test loss 0.5994, total training time 74.12 s, mean epoch training time 4.94 s, test evaluation time 6.18 s, and test throughput 1,619.17 images/s. This is reported separately because its timing summary includes startup effects.

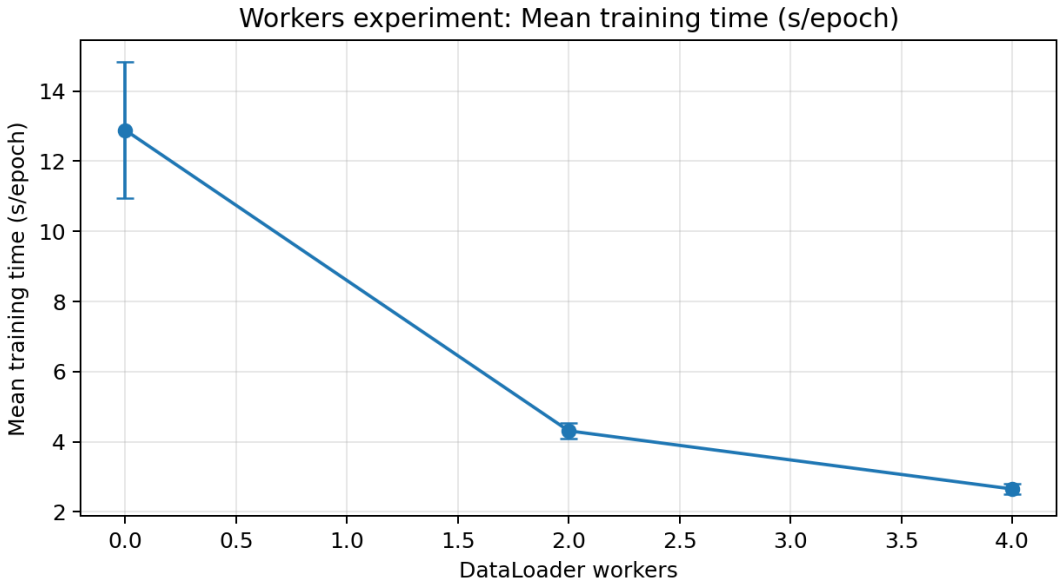
Experiment A. Worker counts 0, 2, and 4 at batch size 128; three seeds and 15 epochs; steady-state timing excludes epoch one.

Experiment B. Batch sizes 64, 128, and 256 with four workers; three seeds and 15 epochs.

6. Results

6.1 Experiment A — DataLoader workers

Workers	Train s/epoch	Response s/epoch	Images/s	Best val. accuracy
0	12.884 ± 1.943	13.623 ± 2.091	3,556 ± 502	79.793% ± 0.329%
2	4.312 ± 0.222	4.600 ± 0.231	10,474 ± 498	79.947% ± 0.662%
4	2.654 ± 0.152	2.837 ± 0.165	17,061 ± 942	79.640% ± 0.374%

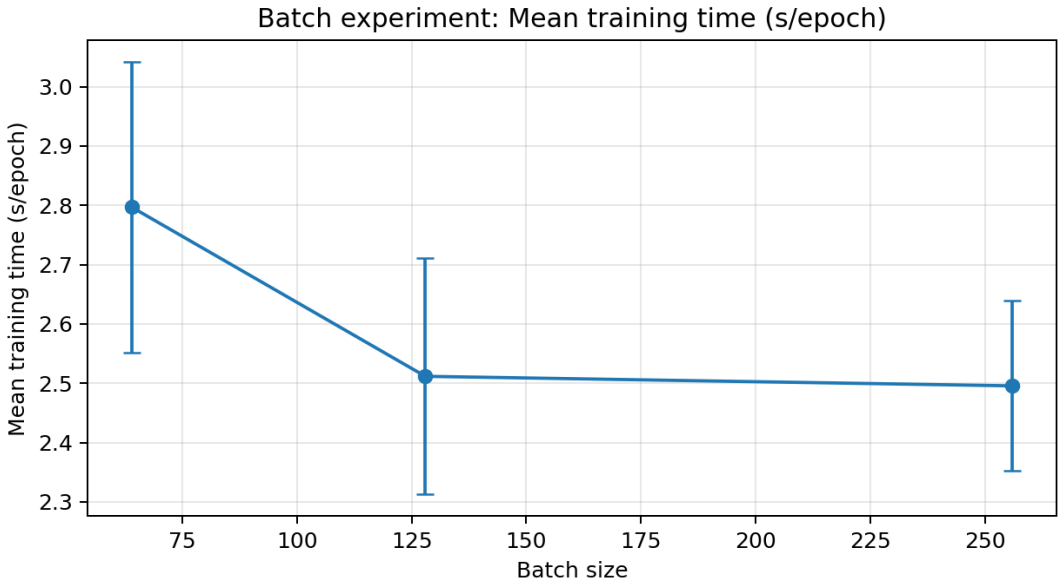


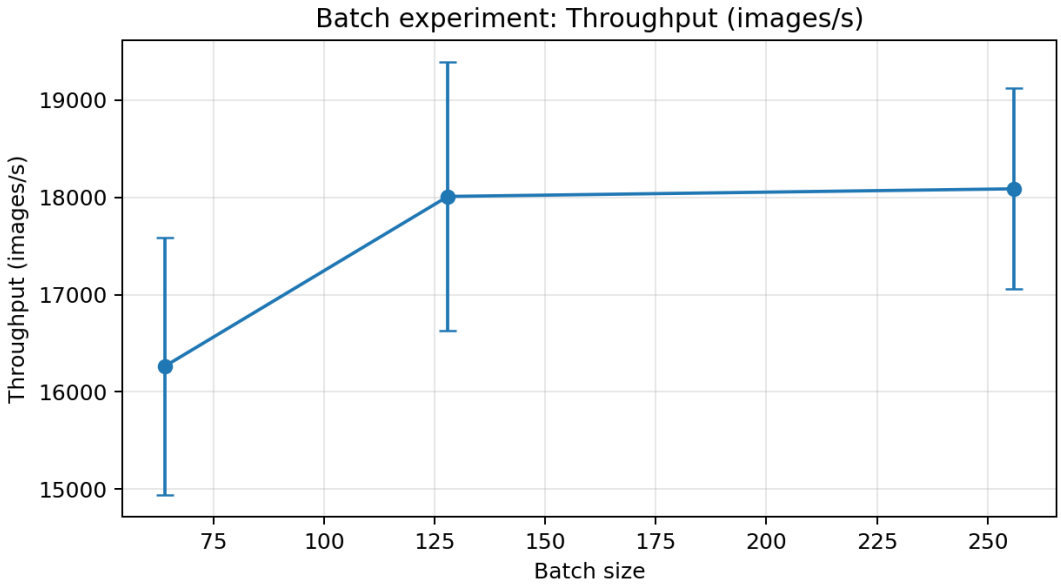


Four workers yielded the shortest mean steady-state epoch time and highest throughput. Comparing two to four workers: 4.3118 / 2.6544 = 1.624× measured time ratio, 38.44% time reduction, and 62.89% throughput increase. Two workers had the highest mean best-validation accuracy in Experiment A.

6.2 Experiment B — Batch size

Batch	Train s/epoch	Response s/epoch	Images/s	Best val. accuracy
64	2.797 ± 0.245	2.973 ± 0.268	16,262 ± 1,325	80.467% ± 0.665%
128	2.512 ± 0.199	2.683 ± 0.216	18,010 ± 1,382	79.640% ± 0.374%
256	2.496 ± 0.144	2.669 ± 0.153	18,088 ± 1,032	79.393% ± 1.115%





Batch size 256 yielded the shortest mean epoch time and highest throughput. Comparing batch size 64 to 256: 2.7967 / 2.4956 = 1.121× measured time ratio, 10.77% time reduction, and 11.23% throughput increase. Batch size 64 had the highest mean best-validation accuracy, about 1.073 percentage points above batch size 256.

7. Performance Metrics

Measured time ratio = baseline time / optimized time. Time reduction (%) = ((baseline – optimized) / baseline) × 100. Throughput change (%) = ((optimized throughput – baseline throughput) / baseline throughput) × 100.

Experiment A (2→4 workers): 1.624× ratio, 38.44% reduction, 62.89% throughput increase. Experiment B (batch 64→256): 1.121× ratio, 10.77% reduction, 11.23% throughput increase.

These are controlled single-GPU epoch-time ratios, not multi-GPU speedup. Distributed parallel efficiency, inter-GPU communication, and all-reduce cost were not measured.

8. Discussion

- Worker count strongly affected epoch timing in this setup.
- The fastest worker setting did not yield the highest validation accuracy.
- Batch sizes 128 and 256 had similar mean epoch times.
- Batch size 64 favoured validation accuracy; batch size 256 favoured throughput.
- Three trials per setting provide limited statistical power.

9. Limitations

- One physical GPU was used; no multi-GPU distributed training was performed.
- Inter-GPU communication, network transfer, all-reduce, and distributed efficiency were not measured.
- Results depend on the hardware, model, software, and data pipeline.
- Warm-up, background activity, caching, and Windows worker-process overhead can affect timings.
- Baseline and repeated-run timing contexts differ; comparisons should use matched steady-state measurements.

10. Conclusion

Four workers achieved the highest throughput in Experiment A and reduced mean epoch time by 38.44% relative to two workers. Batch size 256 achieved the highest throughput in Experiment B and reduced mean epoch time by 10.77% relative to batch size 64. Batch size 64 achieved the highest mean best-validation accuracy. The preferred setting depends on whether throughput or accuracy is the priority. These findings describe single-GPU optimization, not multi-GPU scaling.

11. Reproducibility and Project Files

- src/check_environment.py — environment capture.
- src/dataset.py — DataLoader construction; src/model.py — SmallCNN.
- src/train.py — baseline training.
- src/experiment_workers_repeated.py — repeated worker experiment.
- src/experiment_batch_size.py — repeated batch-size experiment.
- src/analyze_performance.py — measured ratios and throughput changes.
- results/experiment_A_repeated/ and results/experiment_B_batch_size/ — logs, checkpoints, aggregates.
- results/plots/ — experiment plots.

12. References

1. Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images. CIFAR-10: https://www.cs.toronto.edu/~kriz/cifar.html

2. PyTorch documentation, data loading: https://pytorch.org/docs/stable/data.html

3. PyTorch documentation, CUDA semantics: https://pytorch.org/docs/stable/notes/cuda.html

Appendix — Submission checklist

- Verify assignment rubric/template.
- Rerun src/check_environment.py.
- Confirm split and transforms in src/dataset.py.
- Include required logs, checkpoints, aggregate CSVs, and plots.
- Add student/group/instructor details if required.
- Do not claim unmeasured multi-GPU metrics.